大模型应用开发（RAG）技术指南与面试手册

一、 岗位认知与行业现状

目前，AI 行业招聘最看重项目实战能力，其次是证书，最后是学历。对于本科生而言，死磕纯算法岗（如模型训练、微调）竞争极为激烈。更容易入行且岗位需求量大的方向是“AI 应用开发工程师”（或称大模型应用工程师）。

该岗位的核心逻辑是“用模型，而不是造模型”，即利用工程化手段将大模型能力落地成可用的应用。核心技能链包括：Python 工程基础、大模型 API 调用、Prompt Engineering（提示词工程）以及 RAG（检索增强生成）技术。

二、 什么是 RAG？

RAG，全称 Retrieval-Augmented Generation，即检索增强生成。它通过在生成回答之前，从外部知识库中检索相关文档片段，并将这些片段作为上下文提供给大模型，从而生成更准确、更符合事实的回答。

RAG 的核心价值在于解决大模型的“幻觉”问题，并让大模型具备访问私有数据（如公司内部文档、个人笔记）的能力。

三、 RAG 的完整技术链路

一个标准的 RAG 系统包含以下步骤：

1. 文档解析（Document Loading）：读取 PDF、Word、TXT 等格式的文件。
2. 文本切分（Chunking）：将长文档切分成较小的文本块（Chunk）。切分策略非常关键，常见的有按字数切分、按段落切分、按语义切分等。
3. 向量化（Embedding）：使用 Embedding 模型将文本块转换为高维向量，存入向量数据库（如 ChromaDB、Milvus 等）。
4. 检索（Retrieval）：当用户提问时，将问题也向量化，并在数据库中查找相似度最高的文本块。
5. 生成（Generation）：将检索到的文本块作为上下文，与用户问题拼接成 Prompt，发送给大模型生成最终回答。

四、 常见优化方案与面试高频问题

1. 为什么 Chunk_size 不能太大也不能太小？

如果切得太碎（如 100 字），会导致上下文丢失，大模型无法理解完整语义；如果切得太大（如 2000 字），会引入过多无关噪音，浪费 Token，且容易超出大模型的上下文窗口限制。通常需要根据文档类型动态调整。

2. 什么是混合检索（Hybrid Search）？

单纯的向量检索（语义检索）有时无法匹配精确的关键词。混合检索结合了向量检索和传统的 BM25 关键词检索，通过倒数排名融合（RRF）算法合并结果，能大幅提升检索的召回率。

3. 什么是重排序（Reranker）？

初步检索出的 Top-K 文本块可能并非都与问题最相关。引入重排序模型（Reranker）对初步结果进行二次打分和精排，将最相关的文本放在最前面，能有效提升回答准确率。

4. 如何解决大模型的幻觉问题？

除了使用 RAG 引入外部知识外，在 Prompt 中明确限制模型：“如果你不知道答案，请直接说不知道，不要编造。” 同时，可以在返回结果中加入“引用溯源”，标注答案来自哪个文档的哪一页，增加可信度。

五、 求职建议

对于民办本科学生，弥补学历短板的唯一硬通货是“可演示的项目”。简历上不要写“精通各种算法”，而应写“独立完成基于 LangChain 和 ChromaDB 的本地知识库问答应用，支持多格式解析，实现流式输出，并通过云端部署上线”。先入职中小公司或外包岗位积累真实经验，再寻求大厂机会。